

统一片段翻译错误分类分析

本报告把当前可遍历的所有 fragment 合并为一个统一样本池进行分析。统计口径只关心 fragment 类型、完成状态和错误模式。

数据来源与统计口径

- jansi schema：data/java/schemas/deepseek-chat/0.0/jansi
- commons-cli schema：data/java/schemas_evosuite_cleaned_base/deepseek-chat/0.0/commons-cli
- jansi 运行日志：jansi_deepseek-chat_body.log
- commons-cli 运行日志：data/analysis/commons_cli_evosuite_fragment.log
- jansi 明细 CSV：data/java/analysis/jansi_fragment_error_details.csv
- commons-cli 样例明细：data/analysis/commons_cli_fragment_translation_examples.md

全量遍历覆盖

片段类型	数量	全量占比	已完成	类型内完成率	仍未完成	类型内未完成率
方法	686	59.29%	436	63.56%	250	36.44%
字段	454	39.24%	424	93.39%	30	6.61%
静态初始化块	17	1.47%	17	100.00%	0	0.00%
合计	1157	100.00%	877	75.80%	280	24.20%

当前最终状态

最终状态	数量	全量占比	字段	方法	静态初始化块
已完成	877	75.80%	424	436	17
仍未完成	280	24.20%	30	250	0

统一错误模式汇总

错误模式	典型错误行或症状	影响范围	处理优先级
类型、Option、数值宽度、泛型参数不匹配	mismatched types Option<T> 与裸值混用、泛型约束不满足	字段和方法均出现，方法更多	高
Java class literal / 反射类型建模错误	String.class 被翻成 String String.type File.type，或 Class.forName 残留	字段和反射相关方法	高
Java null 语义残留	undeclared identifier 'null'、非可空String上做 null 判断	字段和方法均出现	高

错误模式	典型错误行或症状	影响范围	处理优先级
仓颌集合、字符串、迭代器 API 不匹配	<code>put</code> <code>contains</code> <code>append</code> <code>hasNext</code> <code>substring</code> <code>toString</code> 不存在	方法 fragment 为主	高
Java 标准库或运行时调用未翻译	<code>System</code> <code>Objects</code> <code>Math</code> <code>FileInputStream</code> <code>ProcessBuilder</code> Java 异常类残留	方法和少量字段	高
枚举或静态成员建模错误	Java enum 构造器、枚举字段、静态成员访问被当成普通类成员	枚举和静态成员相关片段	中高
构造器和异常体系不匹配	<code>super(...)</code> 参数不匹配、父类构造调用顺序错误、 <code>throw</code> 非 <code>Exception</code>	异常类构造器和 wrapper 方法	中高
Java 语法残留或仓颌语法错误	三元表达式、 <code>case ?</code> Java <code>cast</code> 、非法 JSON 输出	方法 fragment 为主	中
片段上下文或参数绑定丢失	局部变量、字段、接收者或参数未绑定，如 <code>out</code> <code>codes</code>	方法 fragment 为主	中
框架嵌入问题	完整函数声明被嵌入函数体	少量，但会造成确定性失败	中

统一高频首轮错误行

首轮错误行	观察
<code>error: mismatched types</code>	类型映射、泛型和 Java class literal 建模问题的共同表征。
<code>error: undeclared identifier 'null'</code>	Java nullability 被直接复制到非可空 Cangjie 骨架。
<code>error: undeclared identifier 'System'</code>	Java 运行时 API 未被抽象或替换。
<code>error: undeclared identifier 'Objects'</code>	Java helper API 未翻译，通常需要本地等价实现或直接展开逻辑。
<code>error: 'hasNext' is not a member ...</code>	Java iterator 协议残留，应改为 <code>for-in</code> 或本地 iterator API。
<code>error: 'put'/'contains'/'append' is not a member ...</code>	集合 API 迁移错误，应使用 Cangjie 实际容器方法或下标赋值。

首轮错误行	观察
<code>error: expected a member name after '.' ... keyword 'type'</code>	Java class literal 被错误翻译为 <code>.type</code> 。
<code>error: invalid binary operator '==' on type 'Rune' and 'Struct-String'</code>	字符和字符串字面量混淆，应使用 <code>r'...'</code> 。
<code>error: extra argument given for parameter list ...</code>	构造器或函数签名按 Java 形态调用，未匹配 Cangjie 声明。
<code>--- cjc stderr ---</code>	包级或原始编译器错误，需要回看完整 stderr 才能细分。

阅读说明

- “全量占比”以 1157 个合并 fragment 为分母。
- “已完成/仍未完成”来自当前 schema 中的 `translation_status` 与 `translation` 状态。
- 错误模式按首轮编译失败和代表样例归纳，重点是定位系统性错误来源。
- 代表样例保留原始 fragment 标识；补充样例的完整片段和标准答案见 `data/analysis/commons_cli_fragment_translation_examples.md`。

代表性首轮错误样例

以下样例只作为错误模式例证，不再使用旧的单一来源分母统计。

类型、Option、数值宽度、泛型参数不匹配

样例 1：`jansi.src.main.org.fusesource.jansi.AnsiConsole|61-568:AnsiConsole|561-567:initStreams`

- 片段类型：方法
- 最终状态：仍未完成
- 首轮错误：`error: mismatched types`

Java 源码：

```
static synchronized void initStreams() {
    if (!initialized) {
        out = ansiStream(true);
        err = ansiStream(false);
        initialized = true;
    }
}
```

首次生成的仓颉代码：

```
static func initStreams(): Unit {
    if (!initialized) {
        out = ansiStream(true)
        err = ansiStream(false)
        initialized = true
    }
}
```

```

    }
}

```

样例 2：[jansi.src.main.org.fusesource.jansi.AnsiMain|55-344:AnsiMain|324-335:writeFileContent](#)

- 片段类型：方法
- 最终状态：仍未完成
- 首轮错误：`error: undeclared identifier 'FileInputStream'`

Java 源码：

```

private static void writeFileContent(File f) throws IOException {
    InputStream in__ = new FileInputStream(f);
    try {
        byte[] buf = new byte[1024];
        int l = 0;
        while ((l = in__.read(buf)) >= 0) {
            System.out.write(buf, 0, l);
        }
    } finally {
        closeQuietly(in__);
    }
}

```

首次生成的仓颉代码：

```

private static func writeFileContent(f: File): Unit {
    let in__ = FileInputStream(f)
    try {
        let buf = Array<Byte>(1024, item: 0)
        var l = 0
        while (true) {
            l = in__.read(buf)
            if (l < 0) {
                break
            }
            System.out.write(buf, 0, l)
        }
    } finally {
        closeQuietly(in__)
    }
}

```

仓颉 API 或调用签名不匹配

样例 1：[jansi.src.main.org.fusesource.jansi.AnsiPrintStream|28-96:AnsiPrintStream|34-37:AnsiPrintStream](#)

- 片段类型：方法
- 最终状态：仍未完成

- 首轮错误：`error: extra arguments given for parameter list '()' in call`

Java 源码：

```
public AnsiPrintStream(AnsiOutputStream out, boolean autoFlush, String encoding)
    throws UnsupportedOperationException {
    super(out, autoFlush, encoding);
}
```

首次生成的仓颉代码：

```
public init(out: AnsiOutputStream, autoFlush: Bool, encoding: String) {
    super(out, autoFlush, encoding)
}
```

样例 2：`jansi.src.main.org.fusesource.jansi.Ansi|27-973:Ansi|578-580:cursor`

- 片段类型：方法
- 最终状态：仍未完成
- 首轮错误：`error: extra argument given for parameter list '(Rune, Int32)' in call`

Java 源码：

```
public Ansi cursor(final int row, final int column) {
    return appendEscapeSequence('H', Math.max(1, row), Math.max(1, column));
}
```

首次生成的仓颉代码：

```
public func cursor(row: Int32, column: Int32): Ansi {
    return appendEscapeSequence('H', Math.max(1, row), Math.max(1, column))
}
```

枚举或静态成员建模错误

样例 1：`jansi.src.main.org.fusesource.jansi.AnsiRenderer|164-255:Code|227-230:Code`

- 片段类型：方法
- 最终状态：仍未完成
- 首轮错误：`error: 'n' is not a member of class 'AnsiRenderer'`

Java 源码：

```
Code(final Enum<?> n, boolean background) {
    this.n = n;
}
```

```
        this.background = background;
    }
}
```

首次生成的仓颉代码：

```
init(n: Enum<Any>, background: Bool) {
    this.n = n
    this.background = background
}
```

样例 2：[jansi.src.main.org.fusesource.jansi.Ansi|129-150:Erase|137-140:Erase](#)

- 片段类型：方法
- 最终状态：仍未完成
- 首轮错误：`error: 'value' is not a member of class 'Ansi'`

Java 源码：

```
Erase(int index, String name) {
    this.value = index;
    this.name = name;
}
```

首次生成的仓颉代码：

```
init(index: Int32, name: String) {
    this.value = index
    this.name = name
}
```

Java API 或运行时调用未翻译

样例 1：[jansi.src.main.org.fusesource.jansi.AnsiConsole|61-568:AnsiConsole|174-175:system_out](#)

- 片段类型：字段
- 最终状态：仍未完成
- 首轮错误：`error: undeclared identifier 'System'`

Java 源码：

```
@Deprecated
public static PrintStream system_out = System.out;
```

首次生成的仓颉代码：

```
static var system_out: PrintStream = System.out
```

样例 2：[jansi.src.main.org.fusesource.jansi.AnsiConsole|61-568:AnsiConsole|206-207:IS_WINDOWS](#)

- 片段类型：字段
- 最终状态：仍未完成
- 首轮错误：`error: undeclared identifier 'System'`

Java 源码：

```
static final boolean IS_WINDOWS =  
    System.getProperty("os.name").toLowerCase(Locale.ENGLISH).contains("win");
```

首次生成的仓颉代码：

```
static let IS_WINDOWS: Bool =  
    System.getProperty("os.name").toLowerCase(Locale.ENGLISH).contains("win")
```

Java 语法残留或仓颉语法错误

样例 1：[jansi.src.main.org.fusesource.jansi.AnsiConsole|61-568:AnsiConsole|217-217:IS_CONEMU](#)

- 片段类型：字段
- 最终状态：仍未完成
- 首轮错误：`error: expected ';' or '<NL>', found ')'`

Java 源码：

```
static final boolean IS_CONEMU = IS_WINDOWS && System.getenv("ConEmuPID") != null;
```

首次生成的仓颉代码：

```
static let IS_CONEMU: Bool = IS_WINDOWS && (System.getenv("ConEmuPID") != null)
```

样例 2：[jansi.src.main.org.fusesource.jansi.AnsiConsole|61-568:AnsiConsole|237-445:ansiStream](#)

- 片段类型：方法
- 最终状态：仍未完成
- 首轮错误：`error: expected operator or end of expression, found ':'`

Java 源码：

```

private static AnsiPrintStream ansiStream(boolean stdout) {
    FileDescriptor descriptor = stdout ? FileDescriptor.out : FileDescriptor.err;
    final OutputStream out = new FastBufferedOutputStream(new
FileOutputStream(descriptor));

    String enc = System.getProperty(stdout ? "stdout.encoding" :
"stderr.encoding");
    if (enc == null) {
        enc = System.getProperty(stdout ? "sun.stdout.encoding" :
"sun.stderr.encoding");
    }

    final boolean isatty;
    boolean isAtty;
    boolean withException;
    // Do not use the CLibrary.STDOUT_FILENO to avoid errors in case
    // the library can not be loaded on unsupported platforms
    final int fd = stdout ? STDOUT_FILENO : STDERR_FILENO;
    try {
        // If we can detect that stdout is not a tty.. then setup
        // to strip the ANSI sequences..
        isAtty = isatty(fd) != 0;
        String term = System.getenv("TERM");
        String emacs = System.getenv("INSIDE_EMACS");
        if (isAtty && "dumb".equals(term) && emacs != null &&
!emacs.contains("comint")) {
            isAtty = false;
        }
        withException = false
    } catch (Exception e) {
        withException = true
    }
    // ... 省略 ...
}

```

首次生成的仓颉代码：

```

private static func ansiStream(stdout: Bool): AnsiPrintStream {
    let descriptor = if (stdout) { FileDescriptor.out } else { FileDescriptor.err }
    let out: OutputStream = FastBufferedOutputStream(FileOutputStream(descriptor))
    var enc = System.getProperty(if (stdout) { "stdout.encoding" } else {
"stderr.encoding" })
    if (enc == null) {
        enc = System.getProperty(if (stdout) { "sun.stdout.encoding" } else {
"sun.stderr.encoding" })
    }
    var isAtty: Bool
    var withException: Bool
    let fd = if (stdout) { STDOUT_FILENO } else { STDERR_FILENO }
    try {
        isAtty = isatty(fd) != 0
        let term = System.getenv("TERM")
        let emacs = System.getenv("INSIDE_EMACS")
        if (isAtty && "dumb".equals(term) && emacs != null &&
!emacs.contains("comint")) {
            isAtty = false
        }
    } catch (e) {
        withException = true
    }
    // ... 省略 ...
}

```



```

    } catch (_: Throwable) {
        isAtty = false
        withException = true
    }
    let isatty = isAtty
    var width: AnsiOutputStream.AnsiOutputStream_WidthSupplier
    var processor: AnsiProcessor
    var type__: AnsiType
    var installer: AnsiOutputStream.AnsiOutputStream_IoRunnable
    var uninstaller: AnsiOutputStream.AnsiOutputStream_IoRunnable
    if (!isatty) {
        processor = null
        type__ = if (withException) { AnsiType.Unsupported } else { AnsiType.Redirected
    }

        installer = null
        uninst
// ... 省略 ...

```

片段上下文或参数绑定丢失

样例 1：[jansi.src.main.org.fusesource.jansi.AnsiPrintStream|28-96:AnsiPrintStream|39-41:getOut](#)

- 片段类型：方法
- 最终状态：仍未完成
- 首轮错误：`error: undeclared identifier 'out'`

Java 源码：

```

protected AnsiOutputStream getOut() {
    return (AnsiOutputStream) out;
}

```

首次生成的仓颉代码：

```

protected open func getOut(): AnsiOutputStream {
    return out as AnsiOutputStream
}

```

样例 2：[jansi.src.main.org.fusesource.jansi.AnsiRenderer|45-258:AnsiRenderer|140-142:renderCodes](#)

- 片段类型：方法
- 最终状态：仍未完成
- 首轮错误：`error: undeclared identifier 'codes'`

Java 源码：

```

public static String renderCodes(final String codes) {
    return renderCodes(codes.split("\\s"));
}

```

```
}
```

首次生成的仓颉代码：

```
public static func renderCodes(codes: String): String { return  
renderCodes(codes.split("\\s")) }
```

未声明符号

样例 1：`jansi.src.main.org.fusesource.jansi.internal.JansiLoader` | 61-395:JansiLoader | 237-239:randomUUID

- 片段类型：方法
- 最终状态：仍未完成
- 首轮错误：`error: undeclared identifier 'Long'`

Java 源码：

```
private static String randomUUID() {  
    return Long.toHexString(new Random().nextLong());  
}
```

首次生成的仓颉代码：

```
private static func randomUUID(): String {  
    return Long.toHexString(Random().nextLong())  
}
```

样例 2：`jansi.src.main.org.fusesource.jansi.internal.MingwSupport` | 34-137:MingwSupport | 124-136:getRedirect

- 片段类型：方法
- 最终状态：仍未完成
- 首轮错误：`error: undeclared type name 'ProcessBuilder'`

Java 源码：

```
private ProcessBuilder.Redirect getRedirect(FileDescriptor fd) throws  
ReflectiveOperationException {  
    // This is not really allowed, but this is the only way to redirect the output  
    // or error stream  
    // to the input. This is definitely not something you'd usually want to do,  
    // but in the case of  
    // the `tty` utility, it provides a way to get  
    Class<?> rpi = Class.forName("java.lang.ProcessBuilder$RedirectPipeImpl");  
    Constructor<?> cns = rpi.getDeclaredConstructor();
```

```

        cns.setAccessible(true);
        ProcessBuilder.Redirect input = (ProcessBuilder.Redirect) cns.newInstance();
        Field f = rpi.getDeclaredField("fd");
        f.setAccessible(true);
        f.set(input, fd);
        return input;
    }

```

首次生成的仓颉代码：

```

private func getRedirect(fd: FileDescriptor): Any {
    let rpi = Class.forName("java.lang.ProcessBuilder$RedirectPipeImpl")
    let cns = rpi.getDeclaredConstructor()
    cns.setAccessible(true)
    let input = cns.newInstance() as ProcessBuilder.Redirect
    let f = rpi.getDeclaredField("fd")
    f.setAccessible(true)
    f.set(input, fd)
    return input
}

```

原始 cjc 或包级错误

样例 1：[jansi.src.main.org.fusesource.jansi.AnsiConsole|61-568:AnsiConsole|475-478:out](#)

- 片段类型：方法
- 最终状态：仍未完成
- 首轮错误：--- cjc stderr ---

Java 源码：

```

public static AnsiPrintStream out() {
    initStreams();
    return (AnsiPrintStream) out;
}

```

首次生成的仓颉代码：

```

public static func out_method(): AnsiPrintStream {
    initStreams()
    return out as AnsiPrintStream
}

```

样例 2：[jansi.src.main.org.fusesource.jansi.AnsiConsole|61-568:AnsiConsole|497-500:err](#)

- 片段类型：方法
- 最终状态：仍未完成

- 首轮错误：--- cjc stderr ---

Java 源码：

```
public static AnsiPrintStream err() {
    initStreams();
    return (AnsiPrintStream) err;
}
```

首次生成的仓颉代码：

```
public static func err_method(): AnsiPrintStream {
    initStreams()
    return (AnsiPrintStream) err
}
```

其他编译错误

样例 1：[jansi.src.main.org.fusesource.jansi.Ansi|27-973:Ansi|159-159:detector](#)

- 片段类型：字段
- 最终状态：仍未完成
- 首轮错误：error: expected expression after '=', found '>'

Java 源码：

```
private static Callable<Boolean> detector = () -> !Boolean.getBoolean(DISABLE);
```

首次生成的仓颉代码：

```
static var detector: () -> Bool = { () => !Bool.parse(System.getProperty(DISABLE,
"false")) }
```

样例 2：[jansi.src.main.org.fusesource.jansi.Ansi|27-973:Ansi|174-179:holder](#)

- 片段类型：字段
- 最终状态：仍未完成
- 首轮错误：error: expected '=>' in lambda expression, found 'isDetected'

Java 源码：

```
private static final InheritableThreadLocal<Boolean> holder = new
InheritableThreadLocal<Boolean>() {
    @Override
    protected Boolean initialValue() {
```

```
        return isDetected();
    }
};
```

首次生成的仓颉代码：

```
static let holder: ThreadLocal<Bool> = ThreadLocal<Bool>(initialValue: { isDetected()
})
```

框架嵌入问题：完整函数进入函数体

样例 1：`jansi.src.main.org.fusesource.jansi.Ansi|152-155:Ansi_Consumer|154-154:apply`

- 片段类型：方法
- 最终状态：已完成
- 首轮错误：`error: unexpected modifier 'public' on function declaration in function body`

Java 源码：

```
void apply(Ansi ansi);
```

首次生成的仓颉代码：

```
public func apply(ansi: Ansi): Unit
```

补充代表样例索引

以下样例来自 `data/analysis/commons_cli_fragment_translation_examples.md`，用于补足上方统一错误模式中的更多案例。这里仍只列出 `fragment` 层面的错误类型。

fragment	最终结果	错误类型	具体错因
<code>Option.type__</code>	失败	Java class literal / 类型映射	<code>String.class</code> 被翻成普通值或非法 <code>.type</code> ，应使用 <code>TypeInfo.of<String>()</code> 。
<code>Option.equals</code>	失败	对象语义与强转	Java <code>this == obj</code> 、 <code>instanceof</code> 、强转结构被直接复制，当前骨架中不是合法等价实现。

fragment	最终结果	错误类型	具体错因
<code>Option.getKey</code>	失败	null 语义残留	非可空字符串字段上保留了 Java <code>null</code> 判断。
<code>Option.hashCode</code>	失败	Java helper API 残留	依赖 <code>Objects.hash</code> 和 Java 可空判断，而不是直接基于 Cangjie 字段计算。
<code>OptionGroup.toString</code>	失败	Java iterator 协议残留	使用 <code>hasNext()/next()</code> ，应改为 <code>for-in</code> 或本地 iterator API。
<code>Options.addOption0</code>	失败	集合 API 不匹配	<code>put</code> 、 <code>contains</code> 、 <code>append</code> 等 Java/Python 风格 API 不匹配 Cangjie 容器。
<code>Options.toString</code>	失败	集合渲染方式错误	假设 <code>HashMap.toString()</code> 存在，实际需要显式遍历和拼接。
<code>PatternOptionBuilder.STRING_VALUE</code>	失败	class literal / fragment 边界	输出类级代码或非法 <code>String.type</code> ，而字段 fragment 只需要初始化表达式。
<code>ParseException.ParseException</code>	失败	构造器/异常体系	<code>super(message)</code> 没有匹配生成骨架中的父类构造器形态。
<code>UnrecognizedOptionException.UnrecognizedOptionException</code>	失败	构造器顺序	生成后的类结构使父类构造调用顺序不满足 Cangjie 规则。
<code>Option.clone</code>	成功	Java clone 模式迁移	反馈后从 <code>super.clone()</code> 改为显式构造并复制字段。
<code>Util.stripLeadingAndTrailingQuotes</code>	成功	字符串 API 迁移	反馈后从 <code>substring/slice</code> 改为 Cangjie 区间索引。
<code>Util.stripLeadingHyphens</code>	成功	null 与字符串切片	反馈后用空字符串哨兵和区间索引替代 Java <code>null/substr</code> 逻辑。

fragment	最终结果	错误类型	具体错因
Options.addOptionGroup	成功	集合 API 迁移	反馈后从 append/put 改为 add 和 map 下标赋值。
PatternOptionBuilder.isValueCode	成功	Rune/String 混淆	反馈后从字符串字面量比较改为 r'@' 这类 rune 字面量。